Restaurante con Gestión de Pedidos

Diseño de una solución concurrente

Fase de planificación y diseño



Integrantes

William Balseca

Roberto Falquez

Universidad de Especialidades Espíritu Santo

Sistemas Operativos · Grupo 06

Situación seleccionada

Contexto general del proyecto

¿Qué queremos resolver?

Nuestro proyecto representa la cocina de un restaurante donde llegan varios pedidos al mismo tiempo. Cada pedido necesita ingredientes y debe ser atendido por un cocinero sin que se repita el trabajo ni se usen recursos ya reservados.

Actividades concurrentes

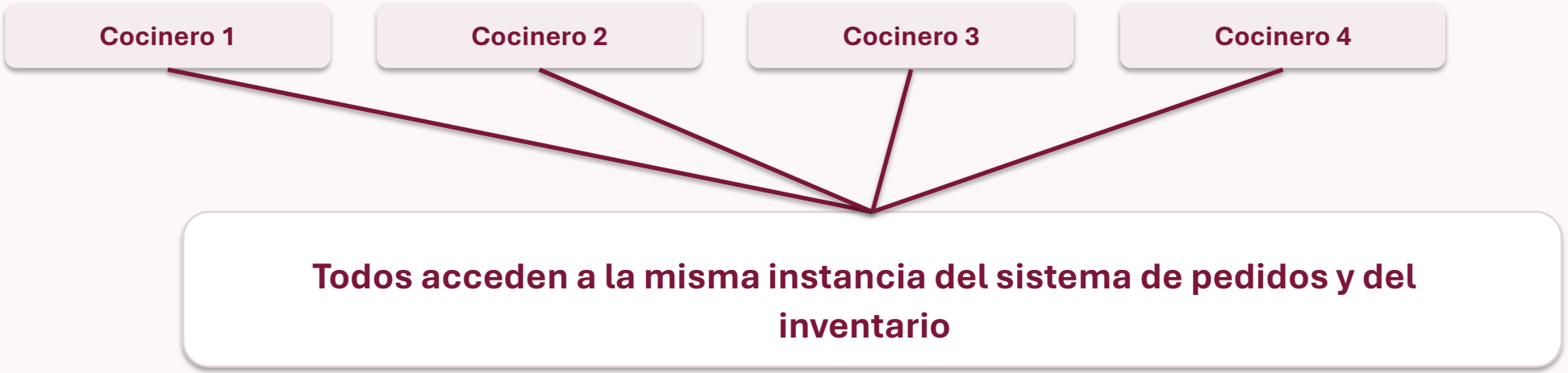
Los cocineros trabajarán al mismo tiempo. Mientras uno selecciona un pedido, otro puede revisar el inventario, reservar ingredientes o retirar un plato ya preparado en masa para entregarlo más rápido.

Idea central

Los hilos del sistema serán los cocineros. Todos compartirán la misma información del restaurante, por eso necesitamos coordinar el acceso al inventario y a los pedidos pendientes.

Los cocineros como hilos

Modelo de cooperación elegido: memoria compartida



¿Por qué este modelo?

Elegimos memoria compartida porque todos los cocineros trabajan dentro de la misma aplicación. Así pueden consultar directamente los datos comunes del restaurante.

¿Qué debe coordinarse?

Debe coordinarse la toma de pedidos, la reserva de ingredientes y el retiro de platos listos, para que no existan repeticiones ni inconsistencias.

Recurso compartido

Qué representa, quién lo usa y por qué puede generar conflictos

Inventario de ingredientes

Representa las cantidades disponibles de pollo, arroz, carne, papas y otros productos asi todos los cocineros lo consultan antes de preparar un pedido.

Platos preparados en masa

Representa las porciones ya listas para entregar, por ejemplo lasaña o arroz con pollo y los cocineros pueden retirarlas para atender pedidos más rápido.

Lista de pedidos

Contiene los pedidos pendientes y Solo un cocinero debe tomar cada pedido para evitar duplicaciones.

Posible conflicto

Si dos cocineros consultan al mismo tiempo el mismo recurso, ambos podrían creer que sigue disponible esto causaría descuentos dobles, cantidades negativas o entrega duplicada del mismo plato.

Representación 1: tipo básico (int)

Diseño conceptual del recurso usando variables enteras

Tipo básico **int**

¿Cómo se interpreta?

En esta alternativa, cada ingrediente se representa con una variable independiente y cada variable guarda una cantidad disponible y se actualiza cuando un cocinero reserva ingredientes para un pedido.

Ejemplo:

```
pollo = 5

carne = 8

arroz = 10

papas = 12
```

Ventaja

Es fácil de entender y resulta útil cuando se quiere controlar pocas cantidades.

Limitación

No agrupa todos los ingredientes en una sola estructura, por lo que se vuelve menos práctico al crecer el sistema.

Representación 2: tipo complejo (dict)

Diseño conceptual del recurso usando un diccionario

Tipo complejo **dict**

¿Cómo se interpreta?

En esta propuesta, los ingredientes se almacenan dentro de una sola estructura, así se puede registrar el nombre del ingrediente junto con su cantidad y administrar mejor toda la información del inventario.

Ejemplo:

```
inventario = {
    "pollo": 5,
    "arroz": 10,
    "carne": 8,
    "papas": 12
}
```

Ventaja

Permite consultar y modificar varios ingredientes desde un mismo recurso compartido.

Por qué nos sirve

Se adapta mejor a un restaurante, porque organiza la información de forma clara y más escalable.

Clases propuestas

Solo diseño: responsabilidades, atributos y métodos

Inventario

Administra las cantidades disponibles de ingredientes. Tendrá los atributos ingredientes y lock, y métodos para consultar, verificar y reservar.

Pedido

Representa cada solicitud del cliente. Guarda el plato, la cantidad, la receta y el estado, y permite consultar o cambiar ese estado.

GestorPedidos

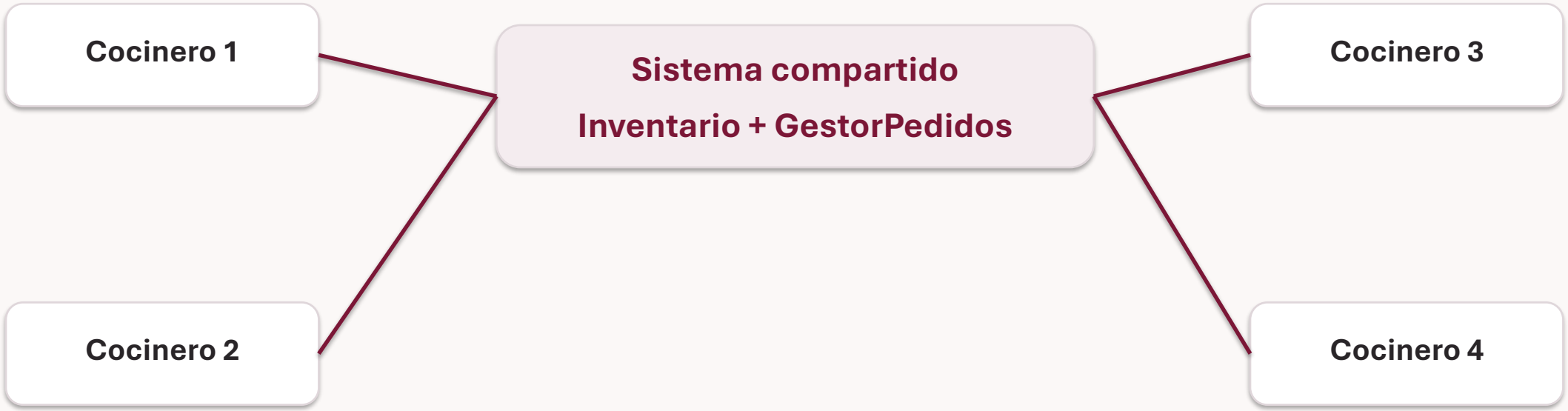
Mantiene la lista de pedidos pendientes. Su función es asignar un pedido a un solo cocinero y evitar duplicaciones.

Cocinero

Representa al hilo de ejecución. Cada cocinero recibe el inventario y el gestor de pedidos para trabajar con la misma información compartida.

Acceso a la misma instancia

Relación entre los hilos y las clases



Interpretación

No se crearán inventarios separados para cada cocinero. Todos trabajarán con la misma instancia, por eso es necesario proteger las operaciones que consultan y modifican la información.

Sección crítica

Operaciones que no deben ejecutarse al mismo tiempo

Operaciones críticas

La sección crítica aparece cuando el cocinero consulta la cantidad disponible, valida si alcanza para el pedido y luego descuenta el recurso. Esto ocurre tanto con ingredientes como con platos preparados en masa.

Ejemplo del conflicto

Si queda una sola porción de lasaña lista y dos cocineros la revisan al mismo tiempo, ambos podrían pensar que sigue disponible y reservarla. El resultado sería una entrega duplicada.

Inconsistencia posible

El sistema podría mostrar cantidades incorrectas, llegar a valores negativos o permitir que un mismo pedido o plato se asigne dos veces, lo cual afecta la lógica del restaurante.

Protección con mutex

Cómo se sincroniza la sección crítica

¿Qué utilizaremos?

Utilizaremos un mutex mediante Lock. Esta opción es adecuada porque queremos que solo un hilo a la vez entre a la parte donde se consulta, valida y modifica el recurso compartido.

¿Dónde se adquiere y libera?

El bloqueo se adquiere justo antes de revisar la disponibilidad del ingrediente o del plato preparado, y se libera después de actualizar la cantidad. Así otro cocinero podrá entrar cuando el cambio ya sea consistente.

1. adquirir lock → **2. consultar y validar** → **3. descontar** → **4. liberar lock**

Comparación de representaciones

Resumen de las dos alternativas de diseño

Tipo básico (int)

Representa una cantidad mediante variables independientes como pollo, carne, arroz y papas. Su principal ventaja es que es fácil de comprender, pero no resulta tan cómodo cuando el sistema necesita manejar muchos recursos.

VS

Tipo complejo (dict)

Reúne varios ingredientes en una sola estructura llamada inventario. Facilita el manejo de la información y hace más clara la relación entre nombre del recurso y cantidad disponible.

Conclusión

Representación seleccionada para la implementación futura

Decisión

La representación que consideramos más adecuada para el proyecto es el diccionario, porque permite manejar mejor el inventario del restaurante y resulta más clara cuando hay varios ingredientes compartidos.

Justificación

El entero sigue siendo útil como ejemplo de tipo básico, pero el diccionario organiza mejor los datos y facilita explicar cómo todos los cocineros accederán al mismo recurso dentro de la solución concurrente.

Cierre

En esta fase solo justificamos el diseño. La programación de las clases y el uso de Lock en Python se desarrollarán después.